

Document Title: Final_report

Multi-Scale PatchTST with Variable-Length Patching and Cross-Channel Attention for Weather Forecasting

Muhammad Kamran Syed^{1, a)}¹ University of Arizona, MSIS-ML, College of Information Science, Tucson, AZ 85721, USA^{a)} s.muhammadkamran@gmail.com

Abstract. Accurate medium-range weather forecasting remains challenging due to the heterogeneous temporal dynamics and strong interdependencies among meteorological variables. Conventional Transformer-based models such as PatchTST [1] employ uniform patching strategies and largely channel-independent processing, which limits their ability to simultaneously model slow atmospheric trends and fast, high-frequency weather fluctuations. To address this limitation, we propose a Multi-Scale Variable-Length PatchTST (MSVL PatchTST) framework specifically tailored for multivariate weather forecasting.

Using a benchmark single-station weather dataset [7] (52,696 time steps, 10-minute resolution), we introduce physics-informed channel grouping, variable-specific patch configurations, dual encoder stacks, cross-group attention, and explicit cyclical time encoding. Slow-evolving thermodynamic variables (pressure, temperature, rainfall) and rapidly changing kinematic variables (wind speed, gusts, rainfall duration) are modeled through dedicated temporal resolutions, enabling scale-aware feature extraction.

Experimental results demonstrate substantial error reduction for pressure, consistent gains for temperature and rainfall, and improved stability in rainfall duration forecasting compared to the baseline PatchTST [1].

The proposed architecture enhances interpretability, preserves computational efficiency (~21M parameters), and provides a scalable design adaptable to operational forecasting systems. Potential applications include meteorological services, renewable energy optimization, agricultural planning, disaster early warning systems, and climate-aware decision-support platforms.

The implementation and experiments are available in a public GitHub repository [8].

1. INTRODUCTION

Weather forecasting plays a critical role in modern society, influencing decision-making across agriculture, transportation, renewable energy management, disaster preparedness, aviation, and climate-sensitive infrastructure planning. Even small improvements in forecasting accuracy can translate into significant economic savings, improved public safety, and better resource allocation. However, accurately predicting weather over short- to medium-term horizons remains a complex, scientific, and computational challenge.

Weather systems are inherently multi-scale and multivariate. Atmospheric pressure and temperature evolve gradually under large-scale thermodynamic processes, while wind speed, gusts, and rainfall can change abruptly due to localized turbulence and convective activity. These variables are strongly interdependent, governed by physical laws such as fluid dynamics and thermodynamics. Capturing both slow-evolving synoptic patterns and fast-changing local dynamics within a unified forecasting model is therefore non-trivial.

Recent advances in deep learning, particularly Transformer-based architectures, have improved long-horizon time-series forecasting. Models such as PatchTST introduce patch-based tokenization to enhance efficiency and long-range dependency modelling. However, most existing architectures assume uniform temporal resolution across variables and often process channels independently. This assumption overlooks the heterogeneous temporal behavior of meteorological variables and limits the model's ability to represent cross-variable physical interactions effectively.

Given the increasing demand for reliable, interpretable, and computationally efficient forecasting systems, there is a clear need for architectures that explicitly account for multi-scale temporal dynamics and inter-variable coupling in weather data. Addressing this gap forms the foundation of this work.

2. LITERATURE REVIEW

This project builds on PatchTST [1], a patch-based Transformer model that improves long-horizon time-series forecasting through patch-level tokenization and channel-independent processing. While PatchTST is computationally efficient, its uniform temporal modelling and lack of explicit cross-variable interaction limit its effectiveness for weather forecasting.

Recent extensions address some of these limitations. CT-PatchTST [3] jointly models channel-time interactions and improves multivariate representation learning but is not tailored to meteorological data and does not support variable-specific patching or physical grouping. Sensorformer [4] introduces crosspatch attention to capture cross-variable and temporal dependencies but lacks multi-scale temporal modelling and mechanisms designed for fast-changing weather variables. InvDec [5] incorporates variate-level decoder attention and shows strong performance on

weather datasets; however, it retains uniform patch lengths and omits physics-aware grouping and explicit time-of-day encoding.

The proposed Multi-Scale Variable Length (MSVL) PatchTST extends these works by introducing variable-dependent patch lengths, channel grouping with group-specific encoders, cross-group attention for inter-variable interaction, and explicit hour-of-day encoding. This design aligns patch-based Transformers with the physical structure and multi-scale dynamics of weather systems.

3. MATERIALS AND METHOD

3.1 Dataset Description

3.1.1 Data Source

The dataset used in this study is the Weather benchmark dataset released with the Autoformer framework (Wu et al., 2021) [2] and widely adopted in long-term time-series forecasting research.

The dataset can be downloaded from the official repository provided by the Autoformer authors [7].

3.1.2 Dataset Scope and Localization

The dataset represents observations collected from a **single fixed meteorological station**, making it a **localized (single location) multivariate time series**. All variables are measured at the same geographic location, and no spatial coordinates, neighbouring stations, or gridded weather products are included.

Consequently:

- The forecasting task is strictly **temporal and multivariate**.
- All dependencies modelled are **cross-variable and temporal**, not spatial.
- The dataset is well suited for evaluating temporal modelling architectures without spatial confounding factors.

3.1.3 Dataset Characteristics

- Total observations: 52,696 time steps
- Sampling frequency: 10-minute intervals
- Temporal coverage: 1 year (01-01-2020 to 31-12-2020)
- Meteorological variables: 20
- Additional engineered temporal features: 2 (hour_sin, hour_cos)
- Total model input channels: 22

3.1.4 Data Split Strategy

The dataset is split chronologically to prevent temporal leakage by dividing time-series data based on its natural temporal order rather than randomly

- Training set: 70%
- Validation set: 10%
- Test set: 20%

3.2 Inclusion Criteria

3.2.1 Channel Group - 1 (Slow changing variables)

Target Variables: Pressure (p), Temperature (T), Rainfall (rain)

- **Temporal behaviors:** These variables exhibit slower, large-scale atmospheric dynamics. Pressure systems evolve over hours to days, temperature follows diurnal and synoptic-scale patterns with gradual transitions, and accumulated rainfall shows persistence with extended dry/wet periods.
- **Correlation structure:** Strong inter-variable dependencies (pressure-temperature coupling through thermodynamics, pressure-rainfall through storm systems).

3.2.2 Channel Group - 2 (Frequently changing dynamics)

Target Variables: Wind velocity (wv), Maximum wind velocity (max.wv), Raining duration (raining)

- **Temporal behaviors:** These variables exhibit rapid, localized fluctuations. Wind speed changes quickly with turbulent atmospheric conditions, maximum wind captures short-lived gusts, and raining (binary/duration) shows abrupt on/off transitions.

- **Correlation structure:** High intra-group correlation (wind speed with max-wind speed metrics).

3.3 Exclusion Criteria

The model is optimized to forecast six primary weather variables: **Air Pressure, Temperature, Wind Speed, Maximum Wind Speed, Rainfall Amount, and Rainfall Duration**. These variables are directly measured, physically meaningful, and operationally relevant, making them suitable targets for focused and interpretable forecasting.

Wind direction is excluded as a prediction target because it is a seasonal feature and needs data spanning multiple years for the location with temperature, pressure and geospatial activity data. In single-station datasets, wind direction is also highly volatile and less predictable than wind magnitude. It is retained as an input feature to provide directional context without complicating model optimization.

3.4 Exploratory Data Analysis (EDA)

3.4.1 EDA was conducted to understand

- Temporal periodicity
- Autocorrelation decay patterns

3.4.2 Diurnal Cycles

Strong daily cycles were observed in temperature, radiation, humidity, and pressure. To encode this periodic behaviour, cyclical hour-of-day encoding was applied:

$$\text{hour_sin} = \sin\left(\frac{2\pi \cdot h}{24}\right), \quad \text{hour_cos} = \cos\left(\frac{2\pi \cdot h}{24}\right) \quad (1)$$

This encoding preserves temporal continuity, avoids artificial discontinuities, and improves learning of daily weather patterns. Such representations are standard in Transformer-based time-series models [6].

3.4.3 Channel Grouping Statistical Evidence

Exploratory analysis of the weather.csv dataset reveals following. This motivated channel grouping, enabling multi-scale modelling.

Table 1. Channel Grouping Statistical Evidence

Feature Group	Autocorrelation Decay	Dominant Frequency
Group - 1	Slow (lag > 24h)	Daily / Weekly cycles
Group - 2	Frequent (lag < 12h)	Hourly / Sub-daily

Autocorrelation Decay: The lag (in hours) at which a variable’s autocorrelation drops below a set threshold (e.g., 0.5), indicating whether it exhibits long-term (> 24h) or short-term (< 12h) temporal memory.

Key insight: The grouping separates thermodynamic/synoptic variables (slow changing) from kinematic/convective variables (frequent, small-scale), enabling scale-specific pattern extraction through differentiated patch lengths.

3.5 Model Architecture

3.5.1 Baseline Model

The original PatchTST architecture applies a uniform patch length and stride across all input variables, treating the entire multivariate time series at a single temporal resolution. It employs a single shared Transformer encoder, which processes patch embeddings without differentiating between variable types or temporal dynamics. Furthermore, the model follows a channel-independent design, where each variable is modelled separately in the embedding space, limiting explicit cross-variable interaction. While computationally efficient, this structure may overlook heterogeneous temporal behaviors present in weather data.

3.5.2 Proposed Model

The proposed Multi-Scale Variable Length PatchTST (MSVL PatchTST) model implements a group-aware Transformer architecture designed to capture heterogeneous temporal dynamics present in weather variables. The model processes multivariate time-series inputs through group-specific encoders and produces forecasts for six meteorological target variables.

Stage 1 – Input Processing:

The model receives 22 input channels consisting of 20 meteorological variables and 2 engineered cyclical time features ($hour_sin$, $hour_cos$).

Input Representation:

The model receives a multivariate time-series input tensor:

$$[X \in \mathbb{R}^{B \times L \times C}] \quad (2)$$

where

- (B) represents the batch size,
- (L) denotes the input sequence length, and
- ($C = 22$) corresponds to the number of input channels (20 meteorological variables plus two engineered temporal features: ($hour_sin$) and ($hour_cos$)).

Before processing, the model applies Reversible Instance Normalization (RevIN) to normalize the input channels and stabilize training across variables with different physical magnitudes. The tensor is then permuted to:

$$[X_{perm} \in \mathbb{R}^{B \times C \times L}] \quad (3)$$

to match the channel-first format required by the encoder modules.

Stage 2 – Parallel Group Encoders:

Each group is processed independently through a dedicated Transformer encoder containing multi-head self-attention layers and feed-forward networks with GELU activation. This enables group-specific temporal resolution modelling. Although each group predicts only its designated target variables, both groups use **all 22 channels as predictor inputs**. This design allows the model to capture cross-variable dependencies such as pressure–temperature coupling and wind–precipitation interactions.

Patch-Based Temporal Tokenization

The temporal input sequence is segmented into overlapping patches using a sliding window defined by patch length (P) and stride (S).

The number of patches is given by:

$$\left[N_p = \frac{L-P}{S} + 1 \right] \quad (4)$$

Each patch contains all channels within a local temporal window and is reshaped as:

$$[X_{patch} \in \mathbb{R}^{B \times N_p \times (C \cdot P)}] \quad (5)$$

A linear patch embedding layer projects the flattened patch into the model embedding space:

$$[X_{embed} = X_{patch} W_p] \quad (6)$$

where

$$[W_p \in \mathbb{R}^{(C \cdot P) \times d_{model}}] \quad (7)$$

and (d_{model}) is the Transformer embedding dimension. The resulting tensor becomes

$$[X_{embed} \in \mathbb{R}^{B \times N_p \times d_{model}}] \quad (8)$$

A learnable positional encoding ($PE \in \mathbb{R}^{1 \times N_p \times d_{model}}$) is then added to preserve temporal ordering.

Transformer Encoder Layers

Each group encoder consists of three stacked Transformer attention blocks.

For a given input embedding (X), the attention mechanism computes:

$$[Q = XW_Q, \quad K = XW_K, \quad V = XW_V] \quad (9)$$

where

$$[W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}] \quad (10)$$

Scaled dot-product attention is applied as:

$$\left[\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \right] \quad (11)$$

where (d_k) represents the dimensionality of the key vectors. This operation enables the model to learn dependencies between temporal patches and between variables.

The attention output is refined through a feed-forward network (FFN) with the following architecture:

$$[d_{\text{model}} \rightarrow 4d_{\text{model}} \rightarrow 4d_{\text{model}} \rightarrow d_{\text{model}}] \quad (12)$$

with GELU activations and dropout for regularization.

Prediction Head

After the Transformer encoder layers, the patch embeddings are flattened:

$$[Z_{\text{flat}} \in \mathbb{R}^{\mathbb{B} \times (N_p \cdot d_{\text{model}})}] \quad (13)$$

A linear projection layer then maps the flattened representation to the forecasting horizon:

$$[Y_{\text{group}} = Z_{\text{flat}} W_o] \quad (14)$$

where

$$[W_o \in \mathbb{R}^{(N_p \cdot d_{\text{model}}) \times (C_{\text{out}} \cdot H)}] \quad (15)$$

Here

(C_{out}) is the number of output variables for the group, and

(H) represents the prediction horizon.

The output tensor for each group is reshaped to:

$$[Y_{\text{group}} \in \mathbb{R}^{\mathbb{B} \times C_{\text{out}} \times H}] \quad (16)$$

Stage 3 – Cross-Group Interaction:

The outputs of both groups are concatenated and passed through a cross-group attention module, allowing inter-variable dependencies (e.g., pressure–wind coupling) to be learned explicitly. A refinement feed-forward network further enhances feature interaction. Finally, RevIN denormalization restores predictions to physical scale, producing six forecast outputs.

A pre-allocated matrix is created:

$$[Y_{\text{all}} \in \mathbb{R}^{\mathbb{B} \times 6 \times H}] \quad (17)$$

Each group fills its assigned output indices:

Group-1 outputs:

$$[Y_{\text{all}}[:, 0:3, :] = Y_{\text{long}}] \quad (18)$$

Group-2 outputs:

$$[Y_{\text{all}}[:, 3:6, :] = Y_{\text{short}}] \quad (19)$$

This ensures that predictions are placed in the correct variable order:

$$[p, T, \text{rain}, \text{wv}, \text{max. wv}, \text{raining}] \quad (20)$$

Finally, the tensor is permuted into the final forecasting format:

$$[Y_{\text{final}} \in \mathbb{R}^{\mathbb{B} \times H \times 6}] \quad (21)$$

Dimensional Consistency

Dimensional consistency during concatenation is ensured through three mechanisms:

- Different patch lengths **do not change the time-series window**. Instead, they change **temporal resolution of tokenization**.
- **Fixed Output Size**: The configuration parameter ($C_{out} = 6$) guarantees a consistent number of predicted channels.
- **Explicit Output Index Mapping**: Each group writes predictions into predefined indices in the output tensor.
- **Sorted Target Alignment**: Encoder outputs are reordered to match the configured target variable ordering before insertion into the final tensor.

These safeguards ensure that predictions from different encoders can be concatenated without dimension mismatches or variable misalignment.

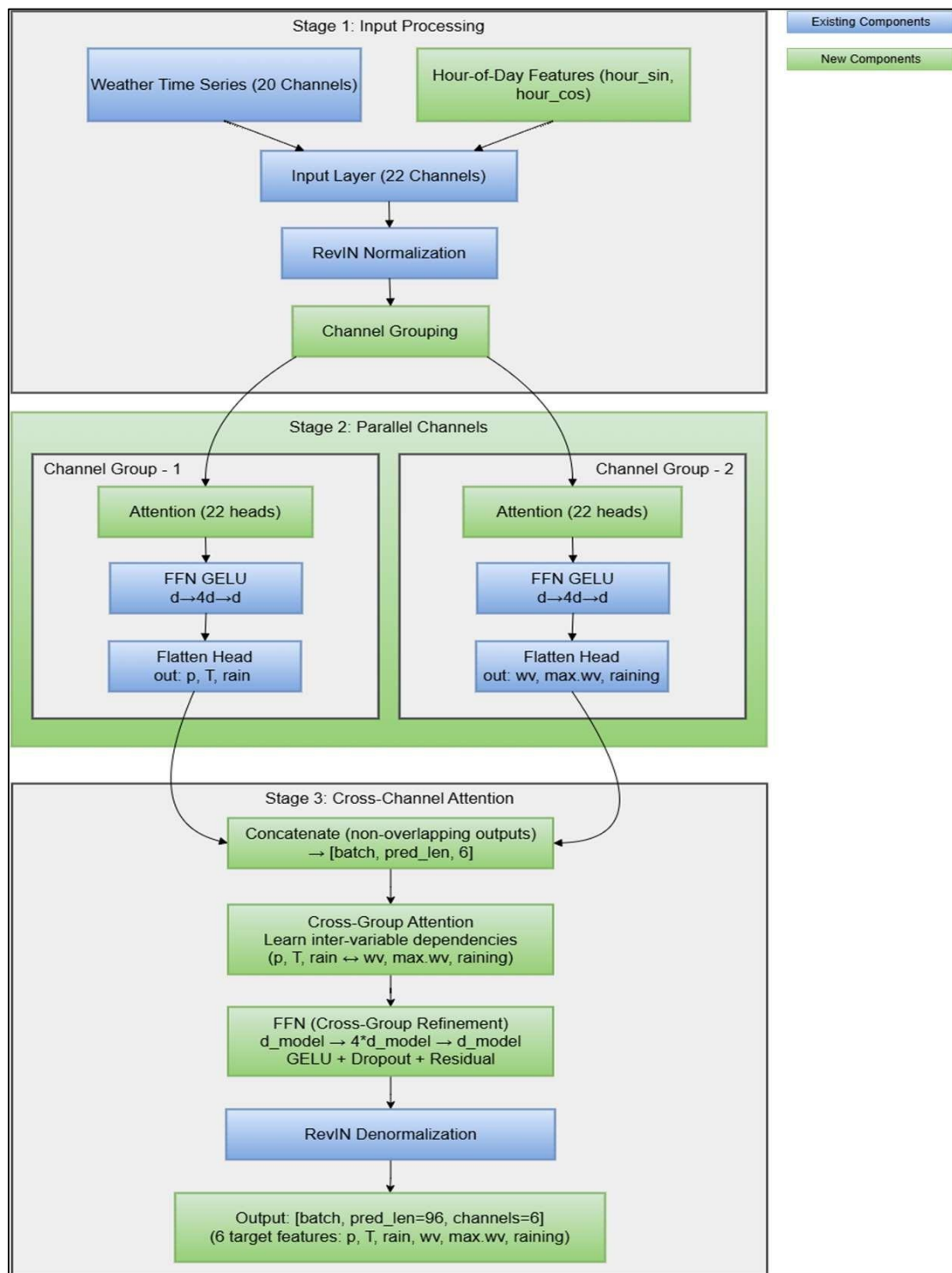


Figure 1. Model Architecture

3.5.3 Channel Grouping and Group Parameters

Table 2. Channel Grouping and Group Parameters

Feature	Channel Group 1	Channel Group 2
Target variables (indices)	p (mbar), T (°C), rain (mm)	wv (m/s), max. wv (m/s), raining (s)
Input channels	All 22 channels (20 weather + 2 hour features)	All 22 channels (20 weather + 2 hour features)
Patch length	8 timesteps	16 timesteps
Stride	4 timesteps	8 timesteps

3.5.4 Hyperparameters

Table 3. Hyperparameters

Parameter	Value	Description
seq_len	336	Input sequence length (56 hours at 10 min resolution)
pred_len	96	Prediction horizon (4 days)
d_model	128	Model embedding dimension
n_heads	8(per-group encoder)	Number of attention heads in group encoders
dropout	0.2	Dropout rate (configurable)
batch_size	32	Training batch size
learning_rate	1e-4	Initial learning rate (AdamW)
train_epochs	100	Maximum training epochs
patience	10	Early stopping patience

3.5.5 Model Parameters

Table 4. Model Parameters

Component	Parameters	Details
Per-Group Encoders	~18M	2 groups × (patch embedding + positional encoding + 3 attention layers + heads)
Patch Embedding	~2K per group	Linear(patch_len, d_model)
Positional Encoding	~5K per group	Learnable (1, n_patches, d_model)
Attention Layers	~8M per group	3 × Custom Multihead Attention with Q/K/V projections + FFN
Cross-Group Attention	~2M	Channel projection + multi-head attention + FFN refinement
RevIN	~88	Affine parameters for 22 channels (mean, std, weight, bias)

3.6 Training and Evaluation

The following section outlines the key training configuration, loss formulation, and evaluation methodology. The settings ensure consistent optimization of behavior across experiments and maintain comparability between model variants.

3.6.1 Target Features

The model predicts six weather-related target variables:

- Channel Group - 1:
 - Pressure (p, mbar)
 - Temperature (T, °C)
 - Rain (mm)
- Channel Group - 2:
 - Wind velocity (wv, m/s)
 - Maximum wind velocity (max. wv, m/s)
 - Raining indicator (raining, seconds)

Evaluation and loss computations are performed only on these six target channels.

3.6.2 Loss Function

The model training uses a loss criterion selected via the designated parameter. For this exercise, **Huber Loss** Function is used.

Huber loss (default $\delta=1.0$) combines the advantages of MSE and MAE, making it particularly suitable for weather forecasting. It provides **robustness to outliers**; extreme weather events (storms, temperature spikes) won't disproportionately dominate the loss, preventing the model from overfitting rare phenomena at the expense of typical conditions. For multi-scale forecasting across variables with different magnitudes (pressure ~1000 mbar, rain ~0-50 mm, wind speed ~0-20 m/s), Huber loss yields **balanced optimization** without requiring extensive manual scaling. Additionally, it maintains **stable gradients** in deep networks (~21M parameters with 3-layer encoders), transitioning smoothly from quadratic (MSE-like) behavior for small errors to linear (MAE-like) for large errors, preventing gradient explosion while maintaining sensitivity to fine-grained prediction errors.

3.6.3 Learning Rate and Scheduler

- Default learning rate: 1×10^{-4}
- Optimizer: AdamW with weight decay 1×10^{-4}
- Scheduler: OneCycleLR using the maximum learning rate equal to the base learning rate.
- EarlyStopping monitors validation performance and saves the best model as a checkpoint. The best checkpoint is re-loaded after training.

3.7 Code Availability

The implementation of the proposed Multi-Scale Variable-Length PatchTST (MSVL PatchTST) framework, including data preprocessing, model architecture, and experimental pipeline, is available at the following GitHub repository: (see [8] in references). The repository is structured to support reproducibility and modular experimentation, enabling independent evaluation of key components such as normalization, temporal patching, channel grouping, and attention mechanisms.

4. RESULTS

4.1 Sensitivity Analysis: Patch Configuration

This table presents experiments performed with various Patch & Strides, to compare and learn the pattern compared to Original PatchTST. The detailed feature-wise results are available in Appendix section 9.3.

Table 5. Comparison table with over-all loss values. First row shows Original PatchTST to compare against the rows with MSVL PatchTST with varying Patch & Stride configurations

Configuration	MSE	MAE	Huber Loss
Original (Baseline) PatchTST (p = 16, s = 8)	0.227529	0.228577	0.072977
Group 1 (p = 32, s = 16), Group 2 (p = 16, s = 8)	0.288631	0.277987	0.100503
Group 1 (p = 8, s = 4), Group 2 (p = 16, s = 8)	0.276922	0.279691	0.096111
Group 1 (p = 16, s = 8), Group 2 (p = 32, s = 16)	0.289495	0.280938	0.100703
Group 1 (p = 16, s = 8), Group 2 (p = 8, s = 4)	0.285887	0.273161	0.098590

Observation: Aggregated metrics such as Mean Squared Error (MSE), Mean Absolute Error (MAE), or Huber loss summarize prediction accuracy across all target variables into a single value. However, in weather forecasting this can be misleading because meteorological variables differ in scale, variability, and predictability. For instance, pressure and

temperature evolve smoothly over time, while wind speed and rainfall can change abruptly. A model may significantly improve predictions for slow-changing variables but perform worse on highly volatile ones, and these differences may cancel out in aggregated results. Consequently, overall metrics may hide important variable-level improvements or degradations. Therefore, feature-wise evaluation is necessary to accurately assess model performance across different weather variables.

Key findings out of sensitivity analysis:

1. **Channel Group - 1 Dominance:** MSVL PatchTST achieves **substantial improvements** on pressure (58-82% error reduction), temperature (3-20% reduction), and rainfall (26-31% reduction). The dedicated encoder successfully captures slow atmospheric dynamics.
2. **Raining Duration Improvement:** Despite being in Group-2, raining (s) benefits from MSVL (20-25% improvement), likely due to its binary/categorical nature being less sensitive to scale-specific modelling.
3. **Patch Length Sensitivity:** Configuration 2 $(8,4) \times (16,8)$ outperforms Configuration 1 $(32,16) \times (16,8)$ for Group-1 features, suggesting shorter patches (8-16) with fine-grained strides (4-8) generalize better. Extremely long patches (32) may lose high-frequency signal components.

4.2 Feature wise Metrics and Plots

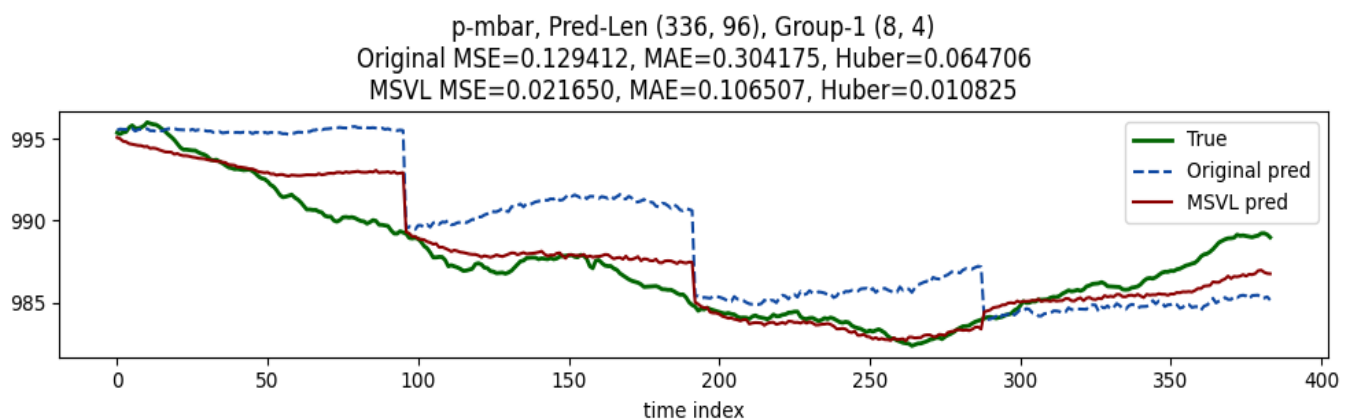
Following results present the Feature wise results for patch configuration

Group 1 ($p = 8, s = 4$), Group 2 ($p = 16, s = 8$)

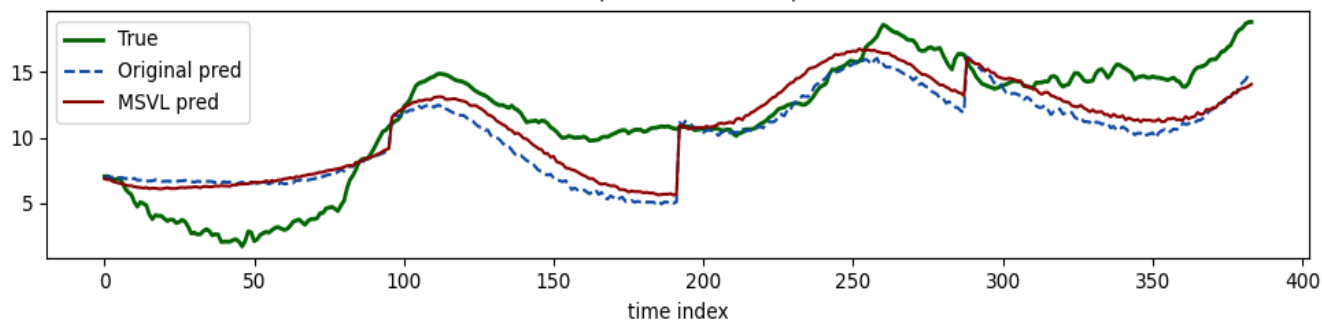
Table 6. Feature Wise Metrics and Plots

Features	Channel Group	Original MSE	MSVL MSE	Original MAE	MSVL MAE	Original Huber	MSVL Huber
p (mbar)	Group-1	0.129412	0.021650	0.304175	0.106507	0.064706	0.010825
T (degC)	Group-1	0.152526	0.111277	0.329664	0.282674	0.076263	0.055638
rain (mm)	Group-1	0.020965	0.020772	0.028434	0.022209	0.009453	0.009357
wv (m/s)	Group-2	0.000419	0.436720	0.015541	0.519642	0.000209	0.202047
max. wv (m/s)	Group-2	0.291926	0.321136	0.414187	0.445616	0.139789	0.152366
raining (s)	Group-2	0.769923	0.749979	0.279464	0.301497	0.147441	0.146430

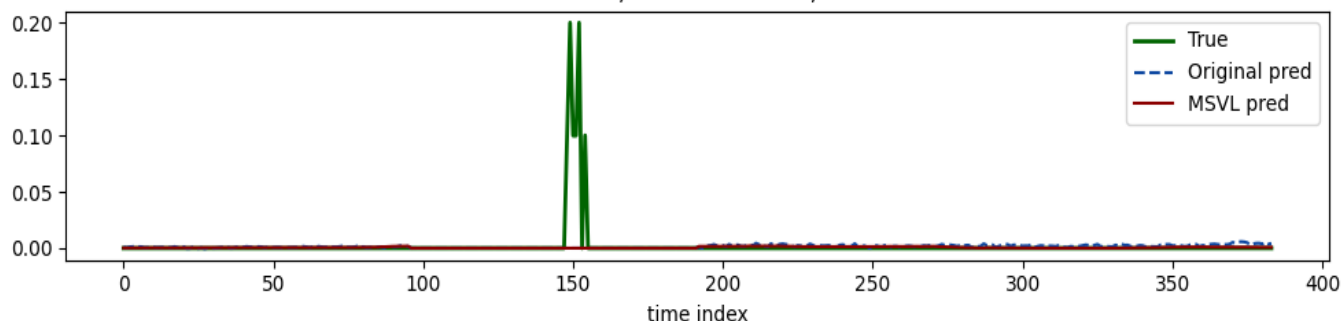
Figure 2. Comparative plots with enhanced results by MSVL PatchTST



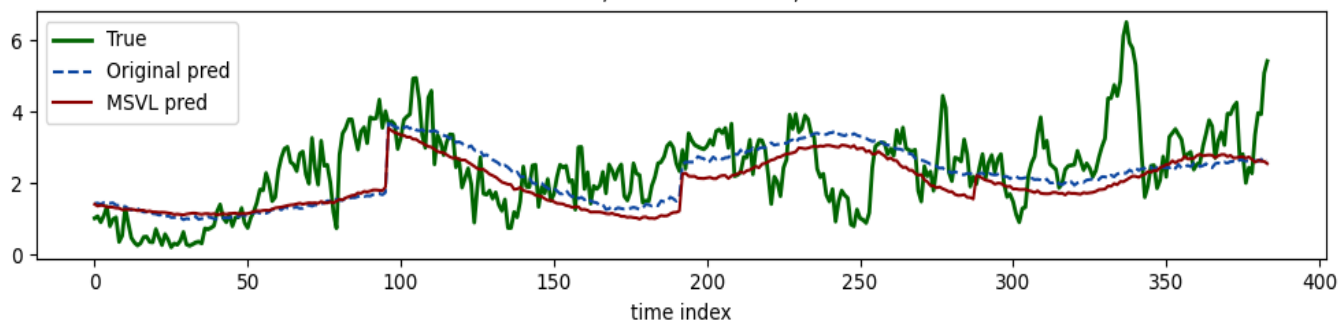
T-degC, Pred-Len (336, 96), Group-1 (8, 4)
 Original MSE=0.152526, MAE=0.329664, Huber=0.076263
 MSVL MSE=0.111277, MAE=0.282674, Huber=0.055638



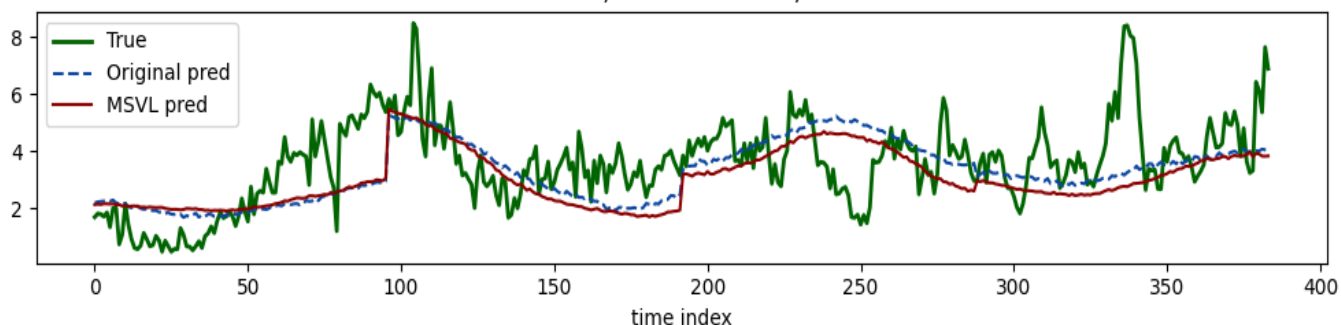
rain-mm, Pred-Len (336, 96), Group-1 (8, 4)
 Original MSE=0.020965, MAE=0.028434, Huber=0.009453
 MSVL MSE=0.020772, MAE=0.022209, Huber=0.009357

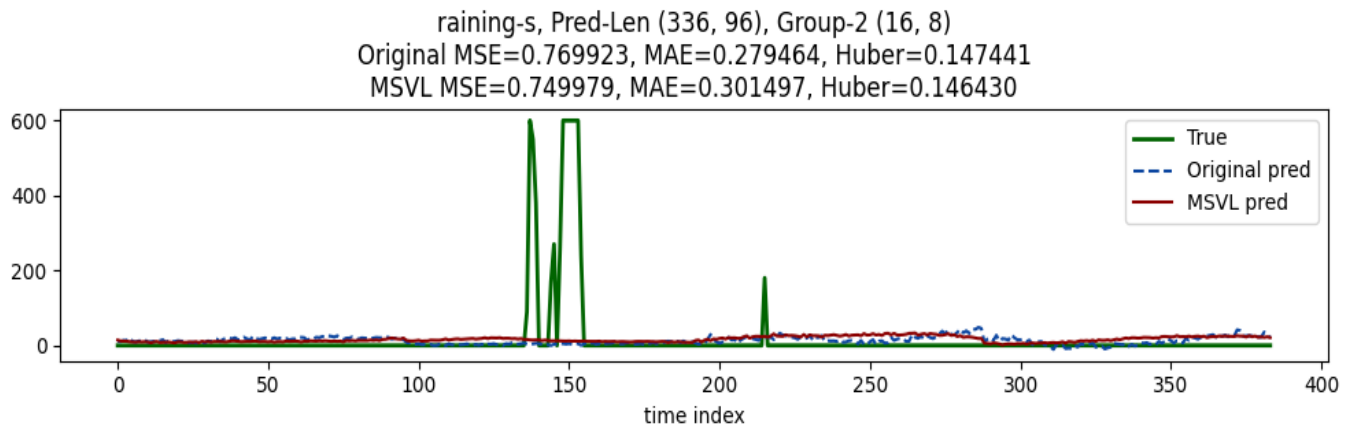


wv-ms, Pred-Len (336, 96), Group-2 (16, 8)
 Original MSE=0.000419, MAE=0.015541, Huber=0.000209
 MSVL MSE=0.436720, MAE=0.519642, Huber=0.202047



max.-wv-ms, Pred-Len (336, 96), Group-2 (16, 8)
 Original MSE=0.291926, MAE=0.414187, Huber=0.139789
 MSVL MSE=0.321136, MAE=0.445616, Huber=0.152366





- **Overall improvement** in Channel Group – 1:
 - Significant error reduction for pressure (p) and temperature (T) variables — **MSE and MAE drop considerably under the MSVL configuration.**
 - Rain (mm) shows roughly **comparable performance** across both setups, indicating minimal benefit from MSVL for low variance, sparse targets.
- **Mixed results** in Channel Group – 2:
 - Wind velocity (wv) experiences error inflation.
 - Maximum wind velocity (max. wv) shows small degradations across all loss types, suggesting moderate instability in short-term dynamics modelling.
 - Raining (s) displays slightly improved results across MSE and Huber, showing better stability than other fast varying variables.

4.3 Ablation Study

4.3.1 Varying stride per channel

To analyse the effect of stride length while keeping patch length fixed (8 for Group-1 and 16 for Group-2), an ablation study was conducted using stride values equal to $\frac{1}{2}$, $\frac{1}{4}$, and $\frac{1}{8}$ of the patch length. Reducing stride increases patch overlap and produces denser temporal sampling. The results show that **smaller strides generally improve prediction accuracy**. The configuration with stride = $\frac{1}{8}$ achieves the lowest MSE (0.2772) and Huber loss (0.0951), indicating improved capture of fine-grained temporal patterns. Although the largest stride yields slightly lower MAE (0.2731), its higher MSE and Huber losses suggest less stable error behaviour. Detailed experiment tables for these configurations are provided in Appendix section 9.3.4 (stride = $\frac{1}{4}$ patch) and Appendix section 9.3.5 (stride = $\frac{1}{8}$ patch).

4.3.2 Moving Cross Attention early in the architecture

To analyse the impact of Cross-Group Attention placement, we compare two configurations: **early fusion**, where cross-group attention is applied before the group-specific encoders, and **late fusion**, where it is applied after independent group predictions. The results show that the **late cross-group attention configuration provides more stable improvements for Group-1 variables**, particularly pressure and rainfall, with substantially lower MSE, MAE, and Huber losses. This suggests that allowing each group encoder to first learn group-specific temporal dynamics improves feature extraction before inter-group interaction occurs. For **Group-2 variables**, the performance difference is smaller, although early fusion slightly improves wind-related metrics. Overall, the findings indicate that **late cross-group attention yields more balanced forecasting performance** across meteorological variables. Detailed experimental results for the early attention configuration are provided in Appendix section 9.3.6.

5. IMPLEMENTATION AND USER BENEFIT

- The model handles different weather patterns more realistically by treating slow-changing and fast-changing variables differently.
- It provides more stable and reliable long-term forecasts, especially for temperature, pressure, and wind trends.
- Related weather variables can interact with each other, helping the model learn meaningful physical relationships.
- The approach reduces the effect of noise in fast-changing data like wind and rainfall.
- Built-in time-of-day information helps the model capture daily weather cycles more accurately.

- The model design makes predictions easier to understand and analyze, improving trust for researchers and forecasters.
- It remains computationally efficient, making it practical for real-world forecasting systems.
- The architecture is flexible and extendable, allowing future improvements without redesigning the entire model.

6. LIMITATIONS AND FURTHER IMPROVEMENTS

The current study also has several dataset and modelling limitations that restrict the generalizability of the results. First, the model is trained using data collected from only a **single weather station**, which limits its ability to learn spatial weather patterns or regional atmospheric variations. Weather systems are **inherently spatial**, and forecasting accuracy could improve if the model were trained on multi-station datasets covering different geographic regions. In addition, the dataset covers roughly **one year of observations**, which means rare or extreme weather events such as severe storms or unusual **seasonal patterns are underrepresented**. Another limitation is that all weather variables are treated with **equal importance during training**, even though certain events such as heavy rainfall or strong winds may be more critical for real-world forecasting applications.

From a modelling perspective, **wind-related variables present additional challenges**. **Wind speed** prediction shows lower accuracy because wind is not governed solely by local time-series behaviour. Instead, it depends on complex atmospheric interactions such as pressure gradients, temperature differences, terrain features, and larger-scale weather systems. Since the current model relies primarily on historical time-series inputs from a single station, many of these external drivers are not captured, making wind dynamics harder to learn. Similarly, **wind direction** was not included as a prediction target because modelling it with standard regression losses is difficult. Wind direction is inherently circular, where values near 0° and 360° represent nearly identical directions.

Another technical issue arises when treating wind direction as a numeric variable. Standard normalization of degrees can distort its cyclic nature. This can mislead neural networks during training. A more appropriate representation would treat wind direction as categorical cardinal directions (e.g., N, NE, E, SE, etc.) or use specialized circular encodings. Future work could therefore improve the model by incorporating multi-location datasets, adding external environmental factors such as geographic information or atmospheric pressure fields, performing deeper hyperparameter tuning, and redesigning wind direction representation using categorical or cyclic encoding strategies.

7. BIBLIOGRAPHY AND REFERENCES

1. Nie, Y., Nguyen, N. H., Sinthong, P., & Kalagnanam, J. (2023). "A Time Series Is Worth 64 Words: Long-Term Forecasting with Transformers." *International Conference on Learning Representations (ICLR 2023)*. <https://arxiv.org/abs/2211.14730>
2. Wu, H., Xu, J., Wang, J., & Long, M. (2021). "Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting." *Advances in Neural Information Processing Systems (NeurIPS 2021)*, 34, 22419–22430. <https://github.com/thuml/Autoformer>
3. Huo, M., Lu, K., Li, Y., Zhu, Q., & Chen, Z. (2025). "CT-PatchTST: Channel-Time Patch Time-Series Transformer for Long-Term Renewable Energy Forecasting." *arXiv preprint arXiv:2501.08620*. <https://arxiv.org/abs/2501.08620>
4. Qin, L., Wang, X., Yang, C., Zou, H., & Zhang, H. (2025). "Sensorformer: Cross-Patch Attention with Global-Patch Compression Is Effective for High-Dimensional Multivariate Time Series Forecasting." *arXiv preprint arXiv:2501.03284*. <https://arxiv.org/abs/2501.03284>
5. Wang, Y. (2025). "InvDec: Inverted Decoder for Multivariate Time Series Forecasting with Separated Temporal and Variate Modelling." *arXiv preprint arXiv:2510.20302*. <https://arxiv.org/abs/2510.20302>
6. Bansal, A., Balaji, K., & Lalani, Z. (2025). "Temporal Encoding Strategies for Energy Time Series Prediction." *arXiv preprint arXiv:2503.15456*. <https://arxiv.org/abs/2503.15456>
7. **Autoformer Dataset**. "Google Drive Link to Datasource" https://drive.google.com/drive/folders/1ZOYpTUa82_jCcXIdTmyr0LXQfvaM9vIy
8. **MSVL PatchTST Code Repo**. "Github Code repo for Multiscale Variable Length PatchTST extension". Implementation and supporting code for the capstone project are maintained in a private GitHub repository to protect the work and are available for verification by authorized reviewers upon request. <https://github.com/s-muhammadkamran/MSVLPatchTST>

8. Tasks & Progress

Table 7. Project tasks and progress.

Task	Sub-task	Dependency	Task-Title	Task-Description	Status
1. Data Preparation, Cleaning & Environment Preparation	1.1	NA	Create Repo	Create the repo on GitHub team for the code and configuration	Completed
	1.2	1.1	Data Audit & Quality Check	Inspect raw weather data, identify missing readings, sensor drift, outliers, and inconsistencies.	Completed
	1.3	1.1	Shared docker container / Conda environment	Shared runtime environment	Completed
	1.4	1.2	Train/Val/Test Split Setup	Chronological dataset splitting ensuring no leakage.	Completed
2. Patch Configuration & Preprocessing	2.1	1.4	Long-Channel Patch Design	Configure patch sizes/stride for slow variables.	Completed
	2.2	1.4	Short-Channel Patch Design	Configure short patches for fast-changing variables.	Completed
	2.3	1.4	RevIN & Scaling Prep	Implement instance normalization per channel.	Completed
	2.4	2.1 to 2.3	Data Loader & Pipeline Testing	Implement and verify PyTorch data loaders.	Completed
	2.5	1.4	Channel Grouping Logic	Implement logic to map variables into long and short channel groups.	Completed
3. Model Development	3.1	2.4	Long-Channel Encoder Implementation	Implement deep attention layers for long-channel encoder.	Completed
	3.2	2.4	Short-Channel Encoder Implementation	Implement encoder for short-channel inputs.	Completed
	3.3	3.1, 3.2	Cross-Group Attention Module	Implement cross-group attention module.	Completed
	3.4	3.1, 3.2	Weather-Feature Integration	Integrate weather feature-based inputs into the architecture.	Completed
	3.5	3.3, 3.4	Output Projection Head	Implement output projection and multi-channel decoder for final forecasts.	Completed
4. Supervised Training & Optimization	4.1	2.4	Baseline Training	Train Original PatchTST baseline model.	Completed
	4.2	3.1 to 3.4	Training New Architecture	Train Weather Variable Length PatchTST.	Completed
	4.3	2.4	Loss Function & Metrics	Implement MSE/MAE and evaluation metrics.	Completed
	4.4	4.2, 4.3	Hyperparameter Tuning	Tune patch sizes, depth, LR, dropout.	Completed
5. Validation, Testing & Ablation	5.1	4.2	Validation Curve Study	Analyze learning curves for under/overfitting.	Completed
	5.2	4.2	Test & Scenario Evaluation	Evaluate model on rain, wind, temperature patterns.	Completed
	5.3	4.2, 4.3	Ablation Studies	Quantify contribution of each model component.	Completed
	5.4	5.2	Error Diagnostics	Analyze variable-wise and diurnal error patterns.	Completed
6. Model Refinement & Integration	6.1	5.1 to 5.4	Design Refinement	Adjust architecture based on evaluation.	Completed
	6.2	5.4	Improve Weather Feature Coupling with groups	Refine weather feature integration with groups	Completed
	6.3	6.1	Efficiency Optimization	Optimize inference, prune layers.	Completed
	6.4	6.1 to 6.3	Final Model Assembly	Export final model for deployment.	Completed
7. Documentation & Presentation	7.1	3.1 to 3.5 6.1	Architecture Documentation	Prepare diagrams comparing original vs new model.	Completed
	7.2	4.1 to 4.4 5.1 to 5.4	Experiment Summary	Document dataset, training procedure, results.	Completed
	7.3	5.1 to 5.4 6.1 to 6.4	Interpretation of Outcomes	Prepare insights and performance summaries.	Completed
	7.4	7.1 to 7.3	Final Report & Presentation	Assemble final report and presentation	Completed

9. APPENDIX

9.1 Hardware Configurations

The project is executed using a **single-GPU workstation** equipped with an **NVIDIA RTX 4060** graphics card (or similar cards). This GPU provides **8 GB of VRAM**, CUDA support, and sufficient tensor-core acceleration to train the proposed **Multiscale Variable Length PatchTST** architecture efficiently for weather data. Typical supervised training runs for the model require approximately **20–40 minutes** per epoch, enabling rapid iteration, hyperparameter tuning, and ablation studies within a practical development cycle.

All experiments will be implemented in **Python (3.12.12)** using the following toolchain:

- i. **PyTorch** for model development, training, and GPU-accelerated operations
- ii. **CUDA/cuDNN** for GPU computation
- iii. **NumPy / Pandas** for data preprocessing and time-series structuring
- iv. **Matplotlib / Seaborn** for visualization and analysis
- v. The environment will run on a Linux container platform with typical configuration:
- vi. **CPU:** Multi-core processor (e.g., Intel i7)
- vii. **RAM:** 16–32 GB system memory
- viii. **Storage:** ~100 GB SSD for dataset handling and model checkpoints
- ix. The computational setup is sufficient for:
 1. Training per-channel encoders and cross-group modules
 2. Running multiple supervised experiments for model comparison
 3. Conducting ablation studies (e.g., patch size, attention depth, weather feature & grouping impact)
 4. Performing performance analysis such as error metrics, plots, and scenario-based evaluations

Overall, the RTX 4060 or similar environment provides a practical and cost-effective platform for developing, experimenting with, and evaluating the proposed multi-scale weather forecasting model.

9.2 Data Dictionary

9.2.1 Original Weather Variables (20)

1. p (mbar) - Air pressure
2. T (degC) - Temperature in degrees Celsius
3. Tpot (K) - Potential temperature in Kelvin
4. Tdew (degC) - Dew point temperature
5. rh (%) - Relative humidity
6. VPmax (mbar) - Maximum vapor pressure
7. VPact (mbar) - Actual vapor pressure
8. VPdef (mbar) - Vapor pressure deficit
9. sh (g/kg) - Specific humidity
10. H2OC (mmol/mol) - Water vapor concentration
11. rho (g/m³) - Air density
12. wv (m/s) - Wind velocity
13. max. wv (m/s) - Maximum wind velocity
14. wd (deg) - Wind direction
15. rain (mm) - Rainfall amount
16. raining (s) - Duration of rainfall
17. SWDR (W/m²) - Shortwave downward radiation
18. PAR (μmol/m²/s) - Photosynthetically active radiation
19. max. PAR (μmol/m²/s) - Maximum PAR
20. Tlog (degC) - Logger temperature

9.2.2 Enhanced Temporal Features (Cyclical Temporal Encoding)

21. hour_sin - Sine-encoded hour of day (captures cyclical daily patterns)
22. hour_cos - Cosine-encoded hour of day (captures cyclical daily patterns)

9.2.3 Dataset Characteristics

- Total observations: 52,696 time-steps
- Sampling frequency: 10-minute intervals
- Temporal coverage: 1 year (01-01-2020 to 31-12-2020)
- Original variables: 20 meteorological variables
- Enhanced variables: 22 variables (20 weather variables + 2 temporal features)

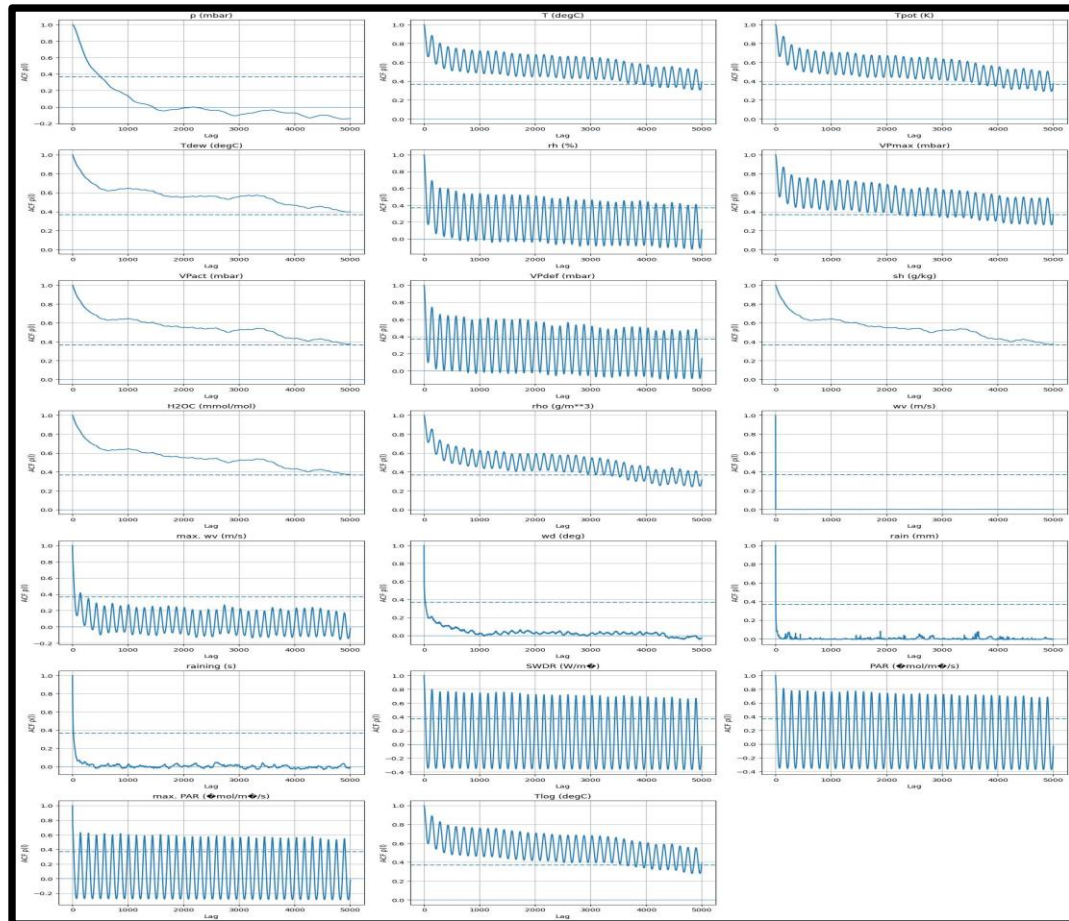


Figure 3. Exploratory Data Analysis

9.3 Experiment results using various Patch configurations

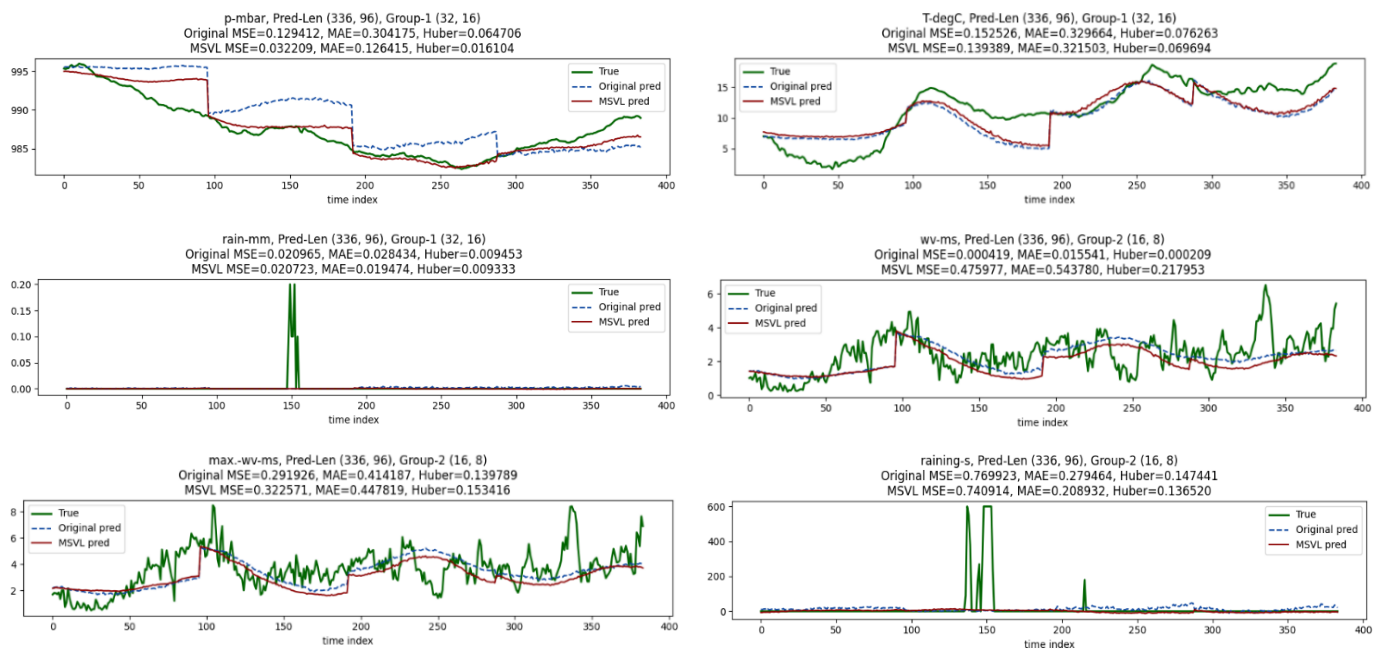
Below section list additional experiments and results conducted with different configurations to tune the hyper-parameters and understand the model enhancement with those results.

9.3.1 Patch Configuration Group-1 ($p = 32, s = 16$), Group-2 ($p = 16, s = 8$)

Table 8. Patch Configuration Group-1 ($p = 32, s = 16$), Group-2 ($p = 16, s = 8$)

Feature	Channel	Original MSE	MSVL MSE	Original MAE	MSVL MAE	Original Huber	MSVL Huber
p (mbar)	Group-1	0.129412	0.032209	0.304175	0.126415	0.064706	0.016104
T (degC)	Group-1	0.152526	0.139389	0.329664	0.321503	0.076263	0.069694
rain (mm)	Group-1	0.020965	0.020723	0.028434	0.019474	0.009453	0.009333
wv (m/s)	Group-2	0.000419	0.475977	0.015541	0.543780	0.000209	0.217953
max. wv (m/s)	Group-2	0.291926	0.322571	0.414187	0.447819	0.139789	0.153416
raining (s)	Group-2	0.769923	0.740914	0.279464	0.208932	0.147441	0.136520

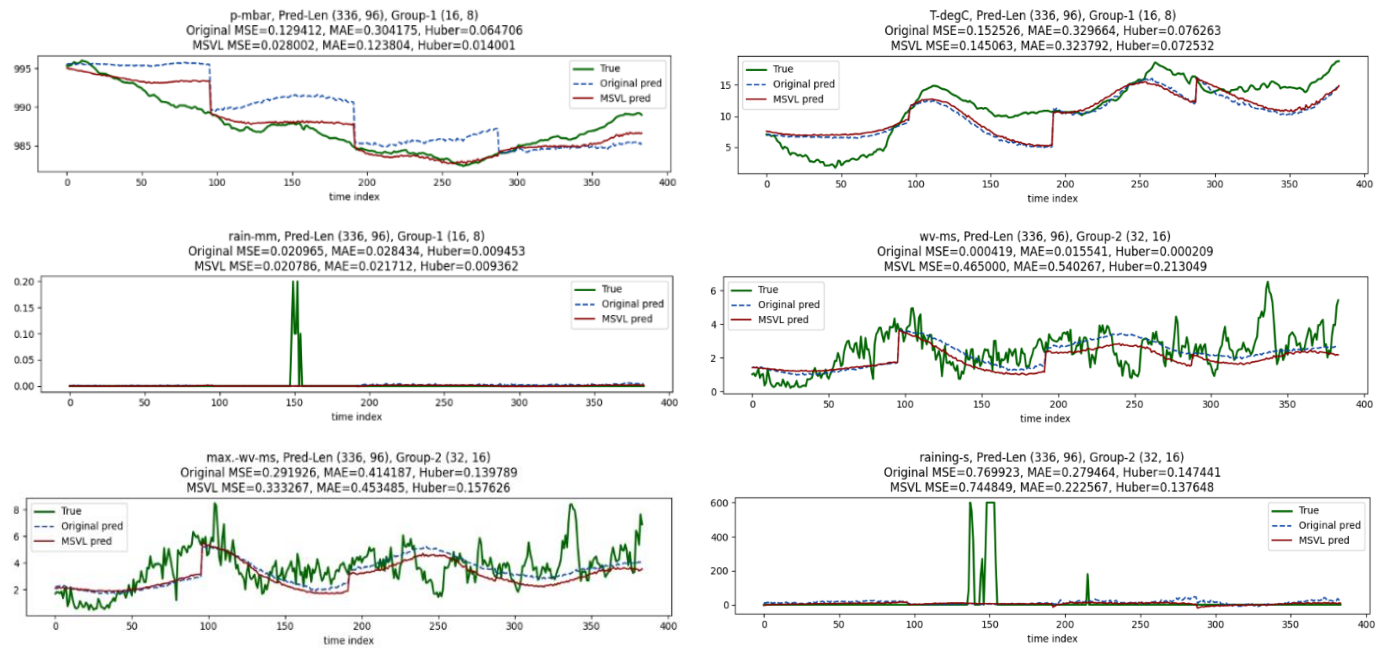
Figure 4. Plots Patch Configuration Group-1 ($p = 32, s = 16$), Group-2 ($p = 16, s = 8$)



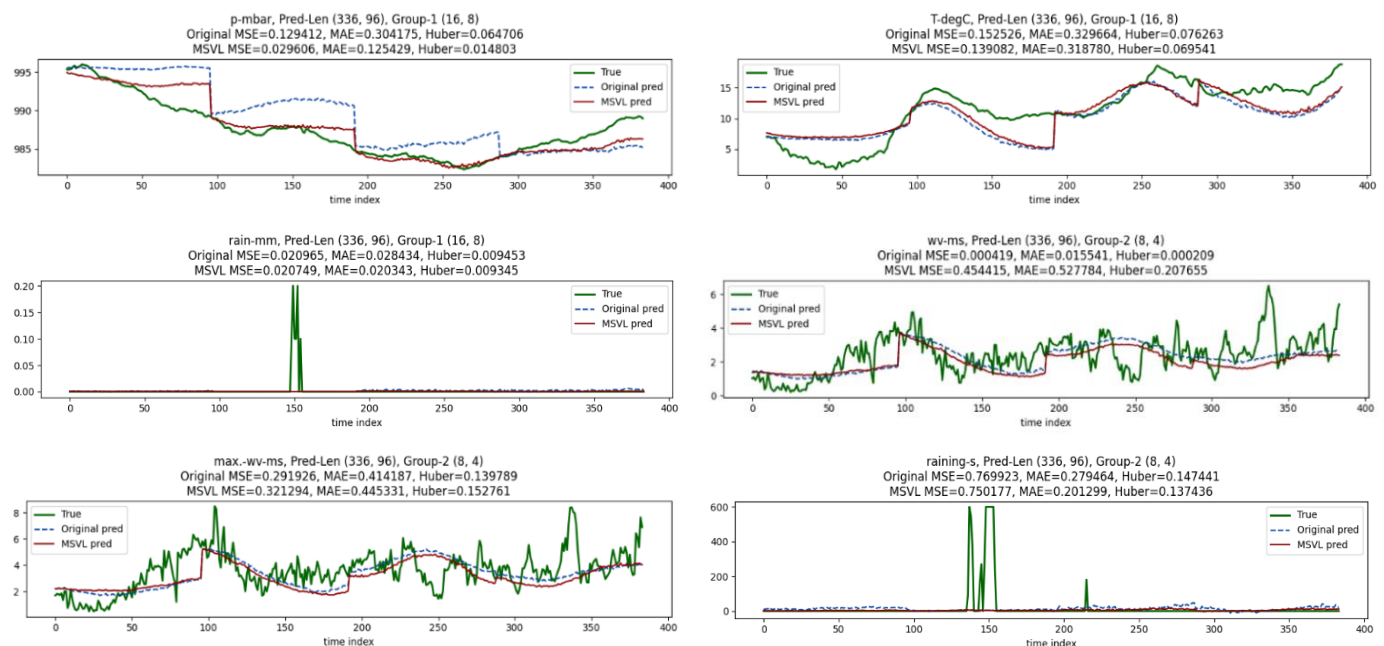
9.3.2 Patch Configuration Group-1 ($p = 16, s = 8$), Group-2 ($p = 32, s = 16$)

Table 9. Group-1 ($p = 16, s = 8$), Group-2 ($p = 32, s = 16$)

Feature	Channel	Original MSE	MSVL MSE	Original MAE	MSVL MAE	Original Huber	MSVL Huber
p (mbar)	Group-1	0.129412	0.028002	0.304175	0.123804	0.064706	0.014001
T (degC)	Group-1	0.152526	0.145063	0.329664	0.323792	0.076263	0.072532
rain (mm)	Group-1	0.020965	0.020786	0.028434	0.021712	0.009453	0.009362
wv (m/s)	Group-2	0.000419	0.465000	0.015541	0.540267	0.000209	0.213049
max. wv (m/s)	Group-2	0.291926	0.333267	0.414187	0.453485	0.139789	0.157626
raining (s)	Group-2	0.769923	0.744849	0.279464	0.222567	0.147441	0.137648

Figure 5. Plot Patch Configuration Group-1 ($p = 16, s = 8$), Group-2 ($p = 32, s = 16$)9.3.3 Patch Configuration Group-1 ($p = 16, s = 8$), Group-2 ($p = 8, s = 4$)Table 10. Group-1 ($p = 16, s = 8$), Group-2 ($p = 8, s = 4$)

Feature	Channel	Original MSE	MSVL MSE	Original MAE	MSVL MAE	Original Huber	MSVL Huber
p (mbar)	Group-1	0.129412	0.029606	0.304175	0.125429	0.064706	0.014803
T (degC)	Group-1	0.152526	0.139082	0.329664	0.318780	0.076263	0.069541
rain (mm)	Group-1	0.020965	0.020749	0.028434	0.020343	0.009453	0.009345
vv (m/s)	Group-2	0.000419	0.454415	0.015541	0.527784	0.000209	0.207655
max. vv (m/s)	Group-2	0.291926	0.321294	0.414187	0.445331	0.139789	0.152761
raining (s)	Group-2	0.769923	0.750177	0.279464	0.201299	0.147441	0.137436

Figure 6. Plot Patch Configuration Group-1 ($p = 16, s = 8$), Group-2 ($p = 8, s = 4$)

9.3.4 Patch Configuration Group-1 ($p = 8, s = 2$), Group-2 ($p = 16, s = 4$)Table 11. Patch Configuration Group-1 ($p = 8, s = 2$), Group-2 ($p = 16, s = 4$)

Feature	Channel	MSE (MSVL stride = 1/2 patch)	MSE (MSVL stride = 1/4 patch)	MAE (MSVL stride = 1/2 patch)	MAE (MSVL stride = 1/4 patch)	Huber (MSVL stride = 1/2 patch)	Huber (MSVL stride = 1/4 patch)
p (mbar)	Group-1	0.021650	0.024409	0.106507	0.118996	0.010825	0.012204
T (degC)	Group-1	0.111277	0.161440	0.282674	0.340864	0.055638	0.080720
rain (mm)	Group-1	0.020772	0.020935	0.022209	0.026862	0.009357	0.009441
wv (m/s)	Group-2	0.436720	0.437208	0.519642	0.512274	0.202047	0.198480
max. wv (m/s)	Group-2	0.321136	0.306092	0.445616	0.432073	0.152366	0.144631
raining (s)	Group-2	0.749979	0.749444	0.301497	0.223172	0.146430	0.138649

The comparison between stride = 1/2 and stride = 1/4 of patch length shows that the larger stride (1/2) generally provides more stable and lower error metrics for most variables. For Group-1 features, pressure and temperature achieve noticeably lower MSE, MAE, and Huber losses with stride = 1/2, indicating that moderate overlap between patches effectively captures slow atmospheric dynamics. Rainfall shows minimal difference between the two configurations due to its sparse and low-variance nature. For Group-2 variables, wind velocity and maximum wind velocity show similar performance across both strides, while **raining duration demonstrates slightly improved MAE and Huber values** with stride = 1/4. Overall, stride = 1/2 offers more consistent predictive accuracy.

9.3.5 Patch Configuration Group-1 ($p = 8, s = 1$), Group-2 ($p = 16, s = 2$)Table 7. Group-1 ($p = 8, s = 1$), Group-2 ($p = 16, s = 2$)

Feature	Channel	MSE (MSVL stride = 1/2 patch)	MSE (MSVL stride = 1/8 patch)	MAE (MSVL stride = 1/2 patch)	MAE (MSVL stride = 1/8 patch)	Huber (MSVL stride = 1/2 patch)	Huber (MSVL stride = 1/8 patch)
p (mbar)	Group-1	0.021650	0.027450	0.106507	0.133542	0.010825	0.013725
T (degC)	Group-1	0.111277	0.122834	0.282674	0.297587	0.055638	0.061417
rain (mm)	Group-1	0.020772	0.021000	0.022209	0.026840	0.009357	0.009477
wv (m/s)	Group-2	0.436720	0.444133	0.519642	0.513936	0.202047	0.201813
max. wv (m/s)	Group-2	0.321136	0.298968	0.445616	0.427869	0.152366	0.142174
raining (s)	Group-2	0.749979	0.748830	0.301497	0.254185	0.146430	0.141986

Comparing stride = 1/2 and stride = 1/8 configurations reveals that the smaller stride (1/8), which creates denser overlapping patches, does not consistently improve performance. For Group-1 variables, stride = 1/2 clearly achieves lower errors for pressure, temperature, and rainfall across all metrics, suggesting that excessive overlap may introduce redundant information rather than improving representation learning. In Group-2 variables, the results are mixed: maximum wind velocity and raining duration show slightly lower errors with stride = 1/8, while wind velocity performs similarly in both settings. **Overall, the stride = 1/2 configuration demonstrates more balanced and reliable forecasting performance** across most meteorological variables.

9.3.6 Patch Configuration Group-1 ($p = 8, s = 4$), Group-2 ($p = 16, s = 8$): Cross Group Attention early in architecture

Table 8. Cross Group Attention early in the model architecture

Feature	Channel	MSE (MSVL Late Cross Attention)	MSE (MSVL Early Cross Attention)	MAE (MSVL Late Cross Attention)	MAE (MSVL Early Cross Attention)	Huber (MSVL Late Cross Attention)	Huber (MSVL Early Cross Attention)
p (mbar)	Group-1	0.021650	0.073723	0.106507	0.205245	0.010825	0.036861
T (degC)	Group-1	0.111277	0.079844	0.282674	0.221854	0.055638	0.039922
rain (mm)	Group-1	0.020772	0.022193	0.022209	0.039921	0.009357	0.010064
wv (m/s)	Group-2	0.436720	0.427610	0.519642	0.494319	0.202047	0.190208
max. wv (m/s)	Group-2	0.321136	0.268729	0.445616	0.384569	0.152366	0.124487
raining (s)	Group-2	0.749979	0.761314	0.301497	0.263943	0.146430	0.145519

This experiment evaluates the effect of **placing the Cross-Group Attention module earlier in the architecture** while keeping the patch configuration fixed at Group-1 (patch = 8, stride = 4) and Group-2 (patch = 16, stride = 8). In this setup, inter-group interactions are introduced before the group-specific encoders complete their independent feature extraction. The goal of this experiment is to assess whether earlier coupling between variable groups can improve the model's ability to capture inter-variable dependencies.

The results indicate that **early cross-group attention generally weakens performance for Group-1 variables**, particularly for pressure and rainfall. Compared with the late-attention configuration, the MSE and Huber losses increase noticeably, suggesting that premature interaction between groups may interfere with the model's ability to learn stable representations of slower atmospheric dynamics. Temperature also shows mixed behaviour, where some metrics improve slightly while others degrade, indicating less consistent optimization.

For Group-2 variables, the impact of early attention is more moderate. Wind velocity and maximum wind velocity show slightly improved MSE and MAE values in some cases, suggesting that faster-changing variables may benefit from earlier interaction with other atmospheric features. However, these improvements are not consistent across all metrics, and raining duration shows only marginal differences between configurations.

Overall, the experiment suggests that **deferring cross-group interaction to later stages of the architecture provides more stable and reliable learning**, particularly for slow-changing meteorological variables. Allowing each encoder to first extract group-specific temporal patterns appears to produce stronger representations before introducing cross-variable dependencies. The quantitative results for this experiment are presented in the table above.